

Architettura degli Elaboratori

Tipologie di indirizzamento e set di istruzioni RISC-V



SAPIENZA
UNIVERSITÀ DI ROMA

Alessandro Checco

alessandro.checco@uniroma1.it

alessandrochecco.github.io

Special thanks and credits:

Andrea Sterbini, Iacopo Masi,

Claudio di Ciccio

[S&PdC] Capitolo 2



Download RARS – RISC-V Assembler and Runtime Simulator

Run speed at max (no interaction) Dec: Hex: Bin: ASCII:

Edit Execute

riscv1.s

```
1 lw t1,0x10010001
```

Line: 1 Column: 17 ☒ Show Line Numbers

Messages Run I/O Program Arguments

☐ Popups
☒ Interactive
☐ Batch

Clear output

Clear input

Runtime exception at 0x00400004: address out of range 0x01001000
-- program is finished running (dropped off bottom) --

Runtime exception at 0x00400004: Load address not aligned to word boundary 0x10010001

Name	Number	Value
zero	0	0x00000000
ra	1	0x00000000
sp	2	0x7fffffc
gp	3	0x10000000
tp	4	0x00000000
t0	5	0x00000000
t1	6	0x10010000
t2	7	0x00000000
s0	8	0x00000000
s1	9	0x00000000
a0	10	0x00000000
a1	11	0x00000000
a2	12	0x00000000
a3	13	0x00000000
a4	14	0x00000000
a5	15	0x00000000
a6	16	0x00000000
a7	17	0x00000000
s2	18	0x00000000
s3	19	0x00000000
s4	20	0x00000000
s5	21	0x00000000
s6	22	0x00000000
s7	23	0x00000000
s8	24	0x00000000
s9	25	0x00000000
s10	26	0x00000000
s11	27	0x00000000
t3	28	0x00000000
t4	29	0x00000000
t5	30	0x00000000
t6	31	0x00000000
pc		0x00400008

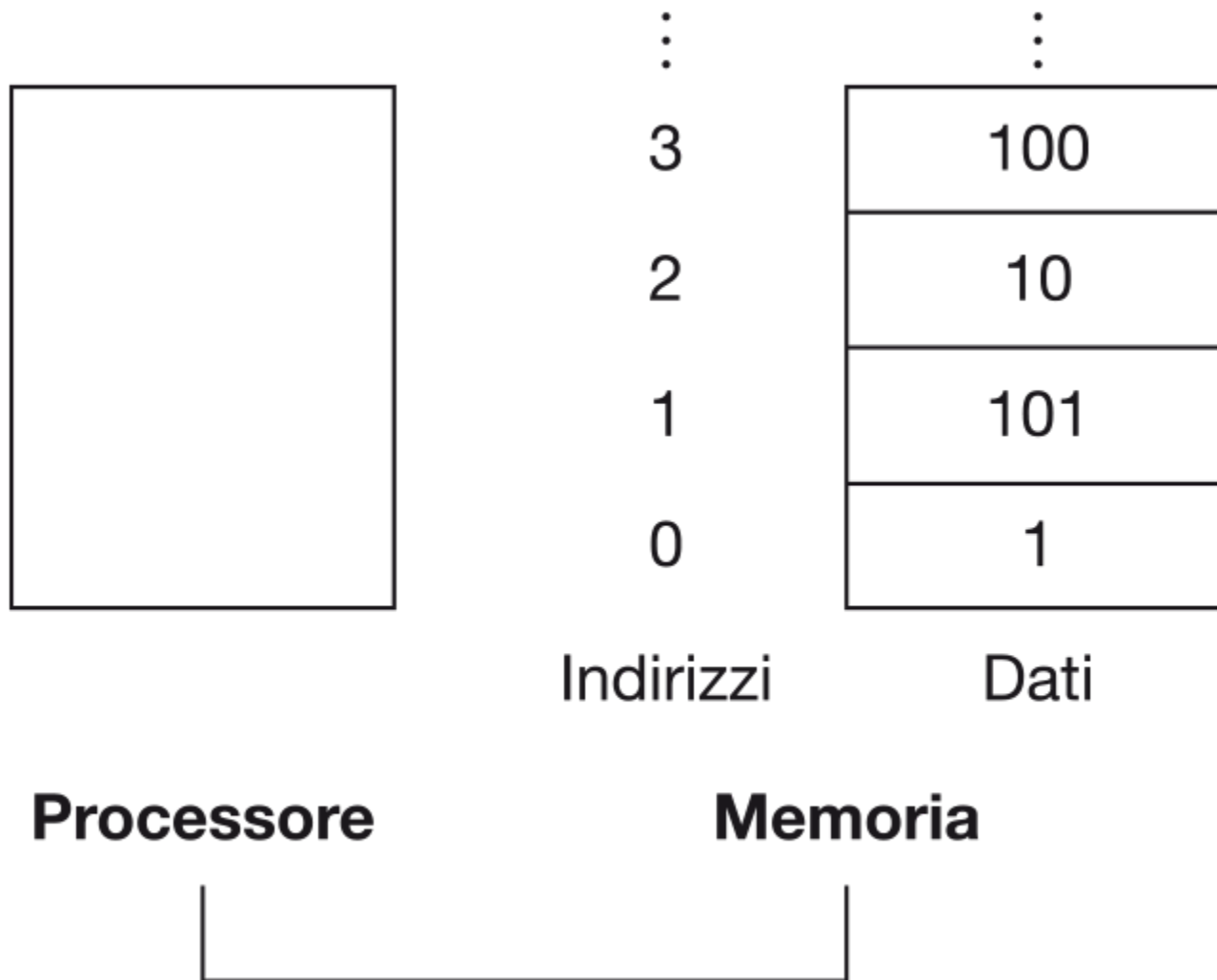
<https://github.com/rarsm/rars/releases/download/latest/rars-flatlaf.jar>

testato su Windows con jdk23

Principi di progettazione

1. La semplicità favorisce la regolarità
2. Minori sono le dimensioni, maggiore è la velocità
3. Un buon progetto richiede buoni compromessi

Memoria



RISC-V non ha vincolo di allineamento per i dati, le word non devono iniziare da indirizzi multipli di 4

Allineamento memoria (esempio)

Sistema 32 bit con vincolo di allineamento

```
#include <stdio.h>

struct NonAllineata {
    char c;
    int i;
};
```

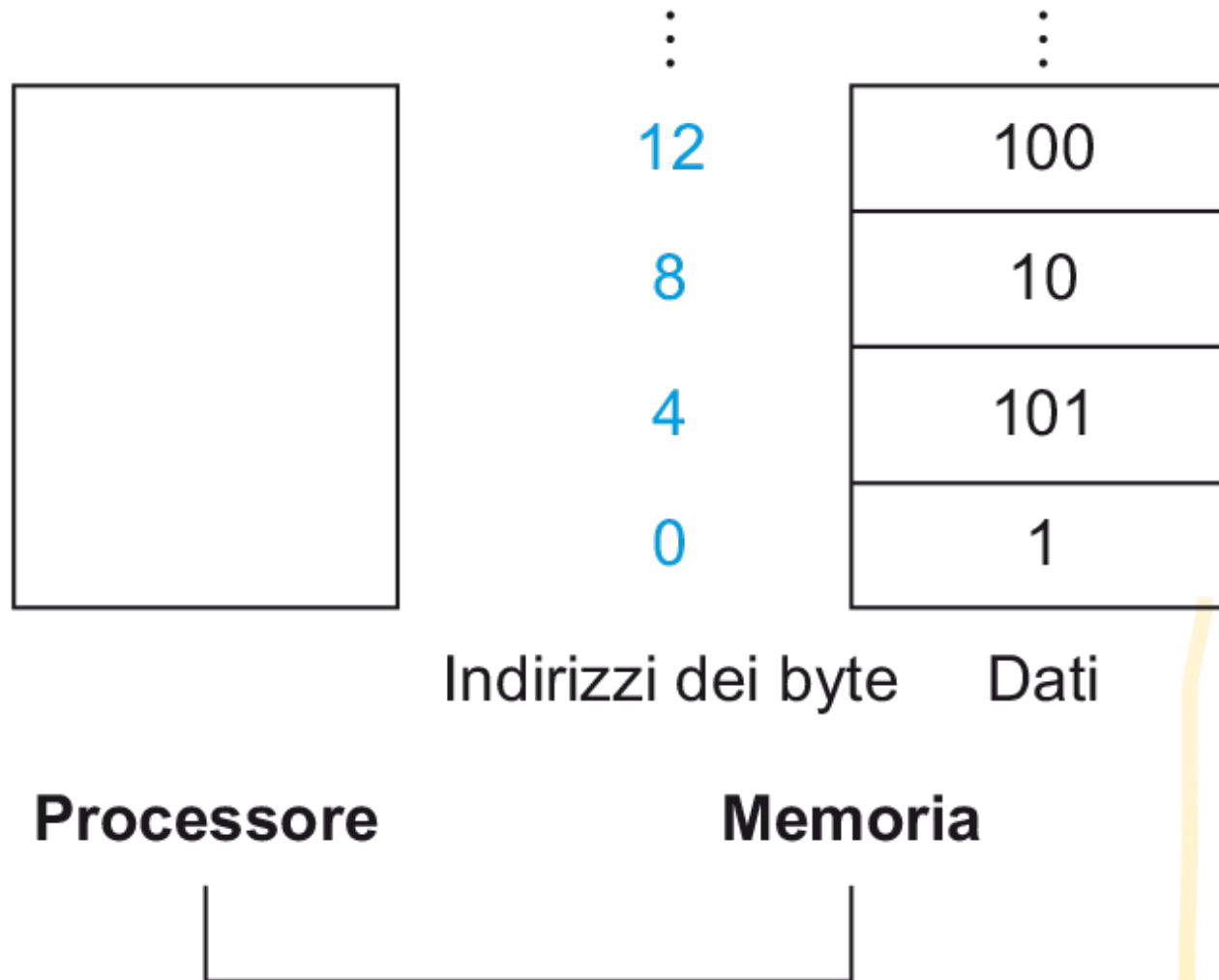
7	i[3]
6	i[2]
5	i[1]
4	i[0]
3	
2	
1	
0	c

```
struct Allineata {
    int i;
    char c;
};
```

4	c
3	i[3]
2	i[2]
1	i[1]
0	i[0]

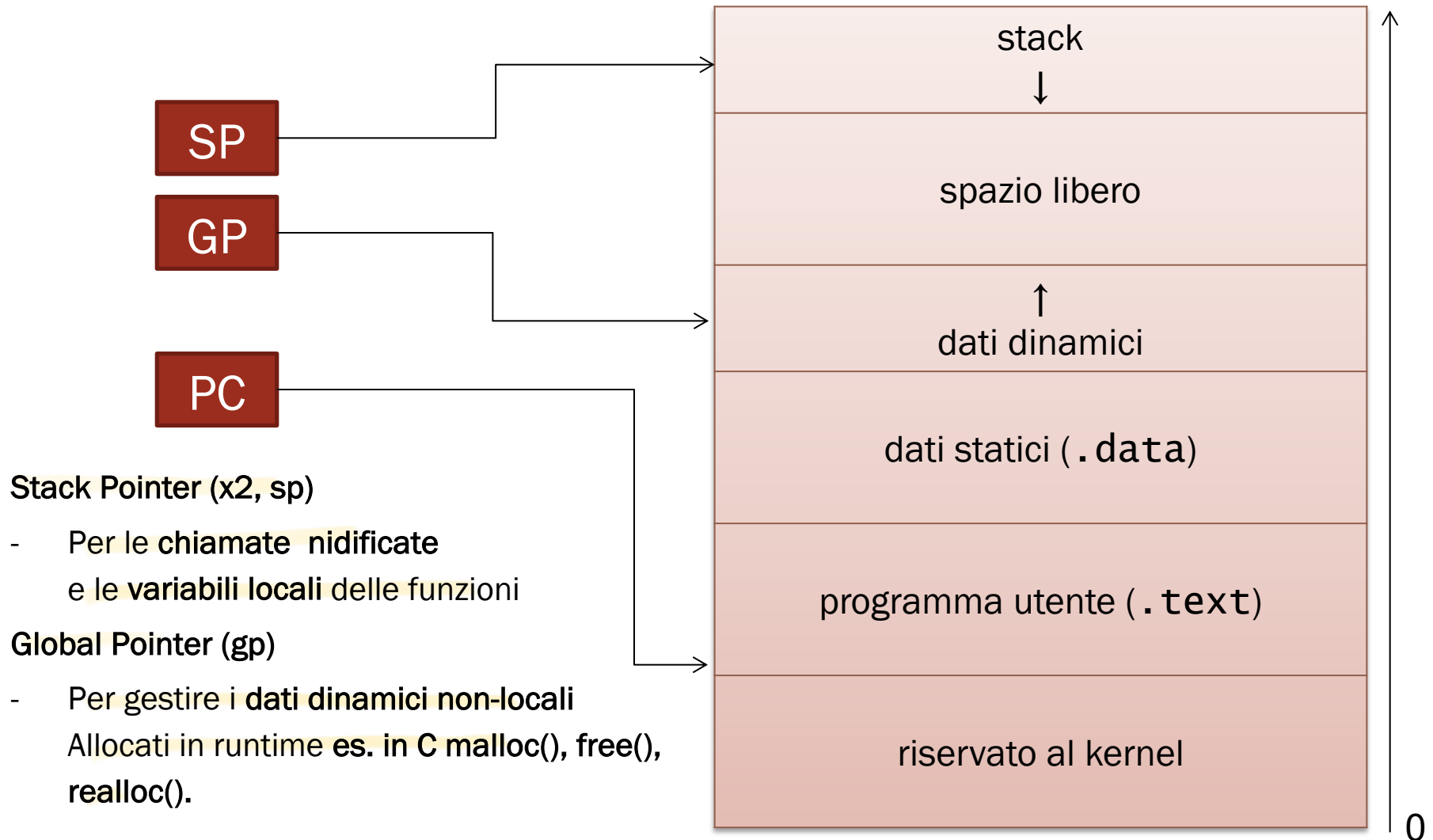
Rars richiede l'allineamento di dati e istruzioni alla parola (indirizzo in byte multiplo di 4)

Memoria (vincolo di allineamento)



Per le istruzioni, RISC-V richiede l'allineamento alla mezza parola (byte multipli di due): combinazione di istruzioni a 16, 32, 64 bit

Organizzazione della memoria



Il set di istruzioni (cosa la CPU «sa fare»)

Le fasi di esecuzione di una istruzione sono:

- **Fetch**/caricamento della istruzione

Dalla posizione indicata dal Program Counter

- **Decodifica**/riconoscimento della istruz.

La Control Unit attiva le parti funzionali necessarie

- **Load**/caricamento di eventuali **argomenti**

A seconda dei modi di indirizzamento (vedi dopo)

- **Esecuzione** della istruzione

In genere da parte dell'ALU

- **Store**/salvataggio del risultato

In memoria o in un registro

- **Aggiornamento del Program Counter**

(contemporaneamente ad altre fasi)

Tipologie di istruzioni:

- **LOAD/STORE**

Trasferiscono dati da/verso la memoria

Es: lb, lbu, lh, lhu, lui, lw, sb, sh

- **Logico/Aritmetiche**

Svolgono i calcoli aritmetici e logici

Es: add, and, sub, subw, or, sll, ...

- **Salти** condizionati e incondizionati

Controllano il flusso logico di esecuzione

Es: beq, bge, bgeu, blt, jal, ...

- **Gestione delle eccezioni/interrupt**

Salvataggio dello stato e suo ripristino

Es: xRET

- **Istruzioni di trasferimento dati**

Non necessarie col memory-mapping

Codifica delle istruzioni (come indicare «cosa fare»)

La **codifica della istruzione** deve indicare:

- Quale operazione va svolta (**Opcode**)

Se necessario quale sotto-operazione

- Quali **argomenti** sono necessari

- Dove mettere il **risultato**

Modi di indirizzamento (come codificare gli argomenti della istruzione):

- **Implicito**

Sorgente/destinazione fissa (p.es. l'AC della IAS)

0 accessi alla memoria

- **Immediato**

Valore costante codificato nella istruzione stessa



0 accessi alla memoria, limitato dalla dim. della istr.

- **Diretto**



1 accesso alla memoria

- **Indiretto**



2 accessi alla memoria

- **A Registro**

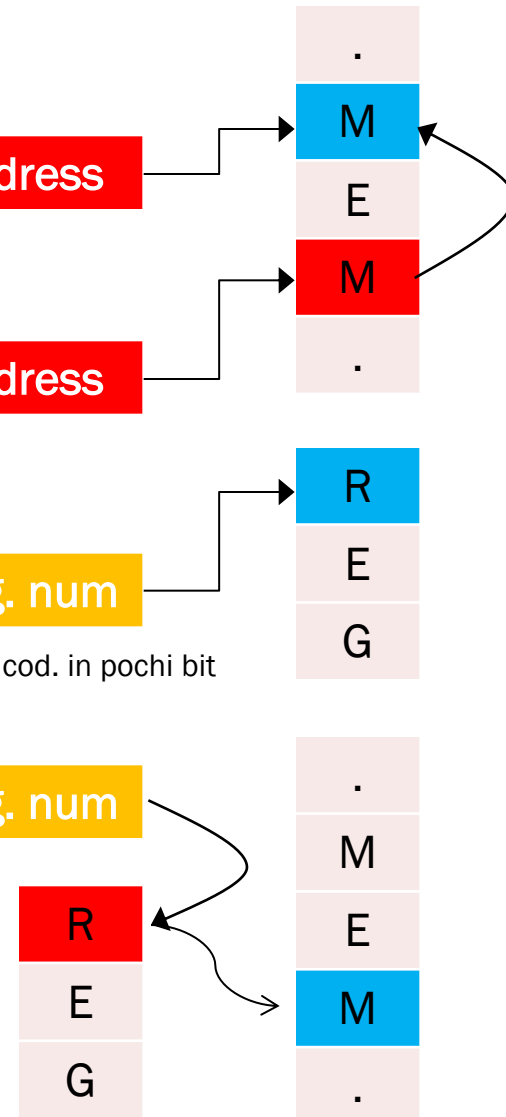


0 accessi alla memoria, cod. in pochi bit

- **A Registro Indiretto**



1 accesso alla memoria



Codifica delle istruzioni (come indicare «cosa fare»)

La **codifica della istruzione** deve indicare:

- Quale operazione va svolta (**Opcode**) (Se necessario quale sotto-operazione)
- Quali **argomenti** sono necessari
- Dove mettere il **risultato**

Modi di indirizzamento (come codificare gli **argomenti** della istruzione):

- **Immediato**

- Valore costante codificato nella istruzione stessa
- 0 accessi alla memoria, limitato dalla dim. della istr.



- **A registro**

- Il registro contiene l'indirizzo di memoria



- **Con Spiazzamento**

- Valore costante codificato nell'istruzione (Spiazzamento)
- Registro contiene il registro con l'indirizzo base dello spiazzamento
- Può indicizzare un byte/mezza parola/parola



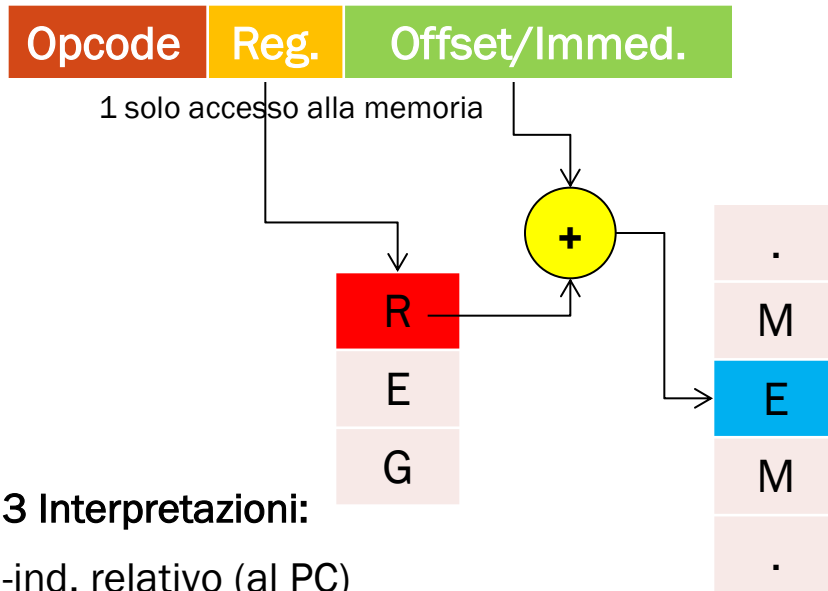
- **Relativo al Program Counter (PC)**

- Valore costante codificato nell'istruzione
- Il valore base è il valore attuale del PC
- Può indicizzare solo parole (istruzioni)



Modi di indirizzamento complessi

- Con spiazzamento (offset)



3 Interpretazioni:

-ind. relativo (al PC)

-ind. relativo a registro base

Il registro è un indirizzo, l'offset una distanza

-indicizzazione di un vettore

Il registro contiene un offset, la parte immediata è l'indirizzo della struttura indicizzata (vettore)

- Altri modi di indirizzamento
(non usati in questo corso):

- post indicizzato

subito dopo il registro viene aggiornato

- pre/post incrementato

il registro è incrementato subito prima/dopo

- Istruzioni condizionate (ARM)

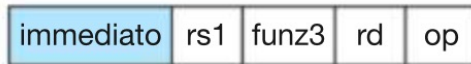
La ALU produce dei condition code (alcuni bit)

Ogni istruzione può essere disattivata

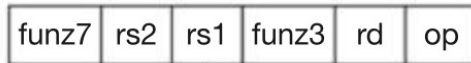
- ...

Indirizzamento – Riepilogo

1. Indirizzamento immediato



2. Indirizzamento tramite registro



Registri

Registro

3. Indirizzamento tramite base e spiazzamento



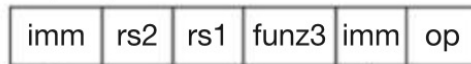
Memoria



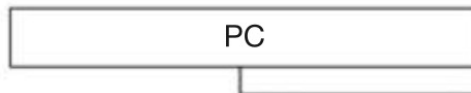
+



4. Indirizzamento relativo al program counter



Memoria



+

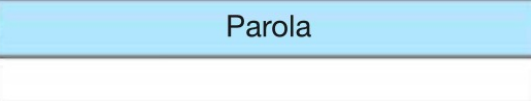


Figura 2.17 Illustrazione delle quattro modalità di indirizzamento del RISC-V. Gli operandi sono evidenziati in blu. L'operando della modalità 3 si trova in memoria, mentre quello della modalità 2 si trova in un registro. Si noti che le varianti delle istruzioni di load e store possono accedere al byte, alla mezza parola o alla parola. Nella modalità 1, l'operando è contenuto nell'istruzione stessa. La modalità 4 viene utilizzata per indirizzare le istruzioni in memoria, aggiungendo un indirizzo ampio al PC. Si noti che un'operazione può utilizzare diverse modalità di indirizzamento: la somma, per esempio, può avere sia un operando immediato (addi) sia tutti gli operandi nei registri (add).

L'architettura RISC-V – utilizzo registri

Nome	Numero del registro	Utilizzo	Da conservare nella chiamata?
x0	0	Costante 0	n.a.
x1 (ra)	1	Registro di ritorno (registro di collegamento)	sì
x2 (sp)	2	Stack pointer (Puntatore a stack)	sì
x3 (gp)	3	Global pointer (Puntatore globale)	sì
x4 (tp)	4	Thread pointer (Puntatore a thread)	sì
x5-x7	5–7	Variabili temporanee	no
x8-x9	8–9	Variabili da preservare	sì
x10-x17	10–17	Argomenti/risultati	no
x18-x27	18–27	Variabili da preservare	sì
x28-x31	28–31	Variabili temporanee	no

Figura 2.14 Convenzioni di utilizzo dei registri RISC-V. Si possono trovare queste informazioni anche nella seconda colonna della scheda tecnica riassuntiva del RISC-V in fondo al libro.

Somma e sottrazione

Testo

Siano

a, b, c, d ed e

variabili contenute rispettivamente in

s0, s1, s2, s3, s4

Scrivere le istruzioni in codice RISC che eseguano questo comando ad alto livello:

$a = (b - c) + (d - e)$



t0



t1

Soluzione

```
sub t0,s1,s2
```

```
sub t1,s3,s4
```

```
add s0,t0,t1
```

Test

```
addi s1,zero,4
```

```
addi s2,zero,3
```

```
addi s3,zero,9
```

```
addi s4,zero,4
```

```
sub t0,s1,s2
```

```
sub t1,s3,s4
```

```
add s0,t0,t1
```

esempio RARS

Movimentazione di dati da/per memoria

Testo

Sia **n** una variabile il cui valore è nella locazione di memoria indicata da **s6** e sia **v** un vettore la cui base è registrata in **s5**. Scrivere le istruzioni in codice RISC-V che eseguano questo comando ad alto livello:
 $v[12] = n + v[6]$

Soluzione

```
lw    t0,24(s5)
lw    t1,(s6) # lw    $t1,0(s6)
add    t0,t1,t0
sw    t0,48(s5)
```

Architetture CISC e RISC (memorandum)

Architettura CISC

(Complex Instruction Set Computer)

- Istruzioni di **dimensione variabile**

Per il fetch della successiva è **necessaria la decodifica**

- **Formato variabile**

Decodifica complessa

- Operandi in memoria

Molti accessi alla memoria per istruzione

- **Pochi registri interni**

Maggior numero di accessi in memoria

- **Modi di indirizzamento complessi**

Maggior numero di accessi in memoria

Durata variabile della istruzione

Conflitti tra istruzioni più complicati

- Istruz. Complesse: pipeline più complicata

Architettura RISC

(Reduced Instruction Set Computer)

- Istruzioni di **dimensione fissa**

Fetch della successiva **senza decodifica della prec.**

- Istruzioni di **formato uniforme**

Per semplificare la fase di decodifica

- **Operazioni ALU solo tra registri**

Senza accesso a memoria

- **Molti registri interni**

Per i risultati parziali senza accessi alla memoria

- **Modi di indirizzamento semplici**

Con spiazzamento, 1 solo accesso a memoria

Durata fissa della istruzione

Conflitti semplici

- Istruz. semplici => pipeline più veloce

L'architettura RISC-V

Word da 32 bit

Spazio di indirizzamento da 32 bit (4GByte) **i 32 Registri della CPU**

indirizzamento con spiazzamento

(include anche gli ind. più semplici)

Interi in complemento a 2 su 32 bit

32 registri di uso generale

- **CPU:** ALU e programma

32 registri + PC

moltiplicazione e divisione intera (opzionale)

Ha accesso alla Memoria

- **CSR:** trap, eccezioni, Virtual Mem.

Registri di Controllo e di Stato

permette di tenere traccia delle eccezioni/interrupt

- **FPU:** per calcoli in virgola mobile (opzionale)

estensioni F (single) o D (double)

32 registri da 32 bit

Ha accesso alla memoria

Registro	Nome	Utilizzo	Chi lo salva
x0	zero	La costante 0	N.A.
x1	ra	Indirizzo di ritorno	Chiamante
x2	sp	Puntatore a stack	Chiamato
x3	gp	Puntatore globale	---
x4	tp	Puntatore a thread	---
x5-x7	t0-t2	Temporanei	Chiamante
x8	s0 / fp	Salvato/puntatore a frame	Chiamato
x9	s1	Salvato	Chiamato
x10-x11	a0-a1	Argomenti di funzione/valori restituiti	Chiamante
x12-x17	a2-a7	Argomenti di funzione	Chiamante
x18-x27	s2-s11	Registri salvati	Chiamato
x28-x31	t3-t6	Temporanei	Chiamante

L'architettura RISC-V a 32bit

Word da 32 bit

Spazio di indirizzamento da 32 bit (4GByte)

indirizzamento con spiazzamento

(include anche gli ind. più semplici)

Interi in complemento a 2 su 32 bit

32 registri di uso generale

- CPU: ALU e programma

32 registri + PC

moltiplicazione e divisione intera (opzionale)

Ha accesso alla Memoria

- CSR: trap, eccezioni, Virtual Mem.

Registri di Controllo e di Stato

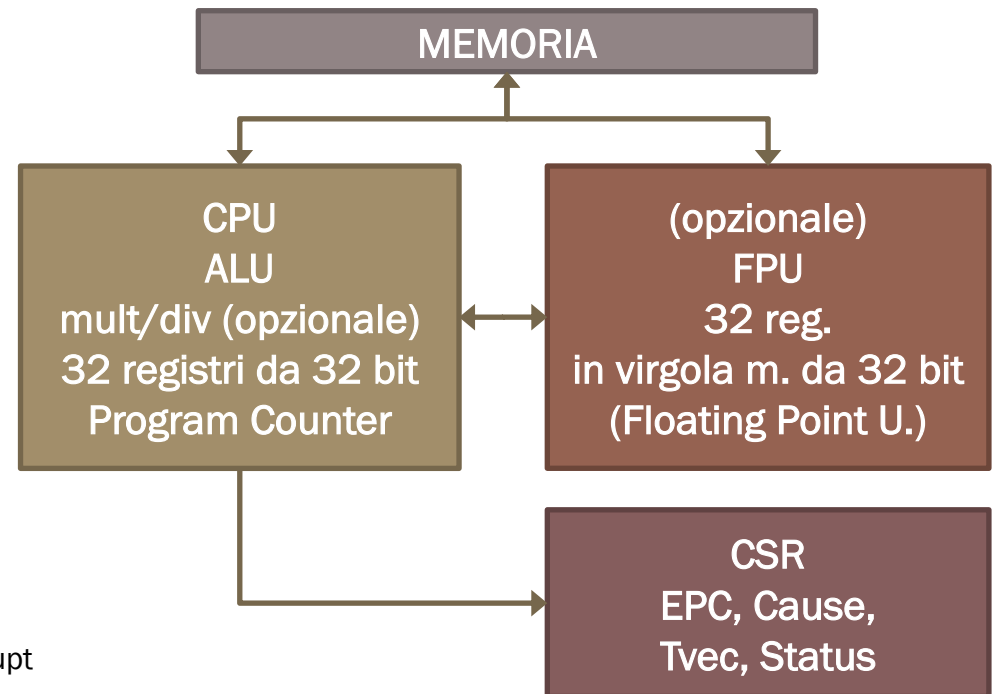
permette di tenere traccia delle eccezioni/interrupt

- FPU: per calcoli in virgola mobile (opzionale)

estensioni F (single) o D (double)

32 registri da 32 bit

Ha accesso alla memoria



L'architettura RISC-V a 32bit Estensioni

Nome simbolico	Descrizione	Numero di istruzioni
I	Architettura di base	51
M	Moltiplicazione/divisione intera	13
A	Operazioni atomiche	22
F	Virgola mobile in precisione singola	30
D	Virgola mobile in doppia precisione	32
C	Istruzioni compresse	36

Figura 2.42 L'architettura dell'insieme delle istruzioni RISC-V è suddivisa in ISA di base, denominata I, e in cinque estensioni M, A, F, D e C.

- L'estensione più semplice ha solo 51 istruzioni
- Tutte le istruzioni di tutte le estensioni sono 184

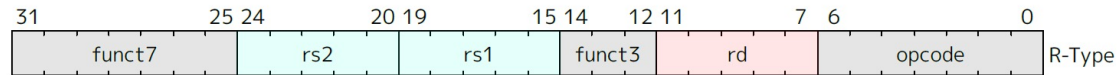
Formato delle istruzioni RISC-V

Istruzioni della CPU tutte da 32 bit con formato molto simile

R-type: (tipo a Registro)

- SENZA accesso alla memoria
- Istruzioni Aritmetico/Logiche

Esempio: **add t0, t1, t2**
sll t0, t1, t2



$t0 \leftarrow t1 + t2$

$t0 \leftarrow t1 \ll (t2 \bmod 32)$

I-type: (tipo Immediato)

- Load
- Somma immediata

Esempi: **addi x1, x2, 1**
lw t1, spiazzamento[B](t0)
slli t0, t1, 2



$t1 \leftarrow t2 + 1$

$t1 \leftarrow \text{MEM}[t0 + \text{Spiazzamento in Byte}]$

$t0 \leftarrow t1 \ll 5$

S-type: (tipo Store)

- Salvataggio registro in memoria

Esempio: **sw x9, 120(x10)**



$\text{MEM}[x10 + 120] \leftarrow x9$

Formato delle istruzioni RISC-V (continua)

Istruzioni della CPU tutte da 32 bit con formato molto simile

U-type: (immediato grande)

31	12	11	7	6	0											
imm[31:12]												rd	opcode			

U-Type

- per movimentare parti immediate con tanti bit (utile anche per salti assoluti)
- può essere usato insieme a addi per costruire costanti a 32 bit in due istruzioni

Esempio: **lui x5, 0x12345** # x5 = 0x12345000

addi x5, x5, 0x678 # x5 = 0x12345000 + 0x678 = 0x12345678

JJ-type: (variante di U)

31	30	21	20	19	12	11	7	6	0							
[20]		imm[10:1]				[11]		imm[19:12]				rd	opcode			

J-Type

- per saltare e salvare il PC +4 in un registro (jump and link)
- Indirizzo di destinazione: PC + immediate x 2 (quindi imm[0] non serve). $\pm 2^{20}$ byte dal PC

Esempio: **jal ra, funzione** #salta a funzione e salva l'indirizzo di ritorno in ra

SB-type: (variante di S)

31	30	25	24	20	19	15	14	12	11	8	7	6	0										
[12]		imm[10:5]				rs2				rs1				funct3		imm[4:1]		[11]		opcode			

B-Type

- due operandi da confrontare e indirizzo di branch nella parte immediata
- indirizzo di destinazione: PC + immediate x 2 (quindi imm[0] non serve). $\pm 2^{12}$ byte dal PC

Esempio: **beq s1,s2,C**

Pseudoistruzioni e etichette in assembly

Pseudoistruzioni:

- Istruzioni non implementati in hardware, ma che possono essere tradotte in istruzioni macchina.
- Modo compatto ed intuitivo per specificare un insieme di istruzioni o istruzioni ricorrenti.
- La traduzione in linguaggio macchina sono effettuate dall'assemblatore
- Es. `mv x10, x11` $\rightarrow$ `addi x10, x11, 0`
 `li, x9, 123` $\rightarrow$ `addi x9, x0, 123`

Etichette:

- Gli indirizzi di salto sono noiosi da calcolare e possono indurre in facili errori
- L'etichetta associa alla parola l'indirizzo dell'istruzione dove è scritta
- L'etichetta potrà essere usata nel codice assembly
- L'assemblatore converte le etichette in salti o in indirizzi
- Le pseudoistruzioni possono usare come argomenti le etichette

Alcuni operatori logici

- slli t0,s1,0x1
- srli t0,s1,0x2
- and t0,s1,s2
- andi t0,x1,0x3
- or t0,s1,s2
- ori t0,s1,0x4

shift / left / logic
shift / right / logic
Logical bitwise “and”
Logical bitwise “and”, I-type
Logical bitwise “or”
Logical bitwise “or”, I-type

Ne faremo un
uso particolare

Ideale per
bitmask

E il NOT? Per mantenere il formato di istruzioni a due operandi, (v. [principio di progettazione 1](#)):

NOT(A) = A XOR 1
pseudoistruzione: not t0, s1
viene tradotta in: xori t0, s1, 0xFFFFFFFF

Esercizio: siano

u e v

due variabili salvate nei registri
s0 e s1

rispettivamente. Si assume u positivo.

Scrivere il codice RISC-V che esegua:

$$v = u \times 256$$

Domanda: da quale valore di u in poi il
risultato non corrisponde a quello atteso?

Come si comporta RISC-V in quel caso?



Comparazione e salto condizionato

- beq s1,s2,C
- bne s1,s2,C
- bge s1,x0,C
- bge x0,s1,C
- blt s1,x0,C
- bgt s1,x0,C

Branch on equal

Branch on not equal

Branch on less-than or eq. 0

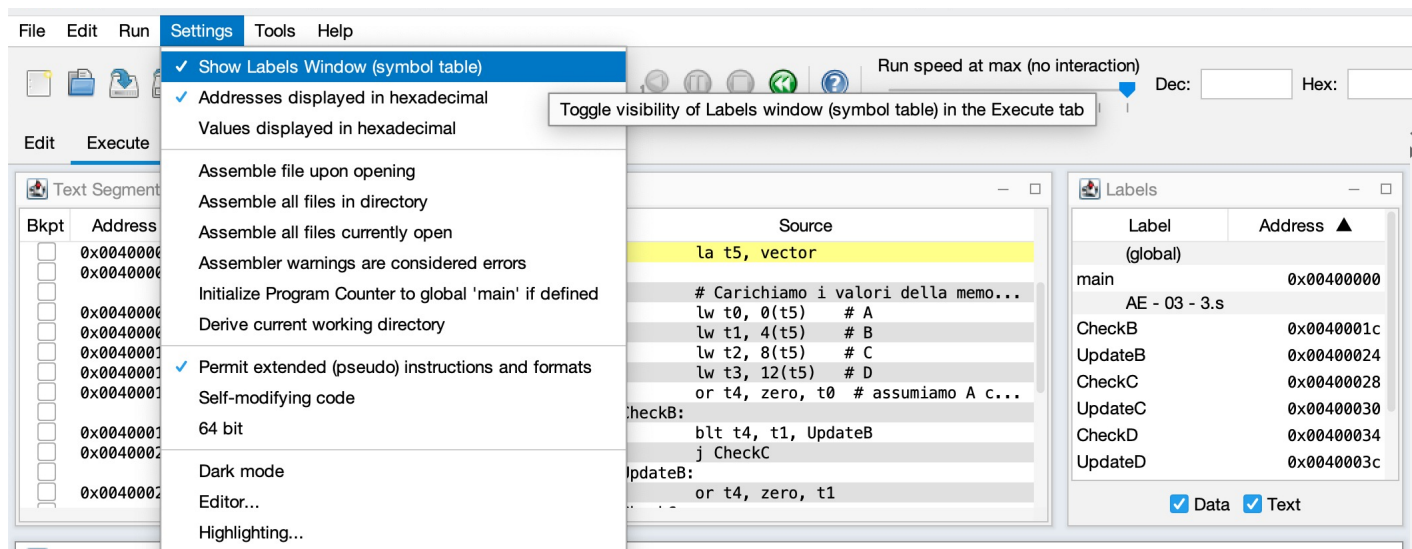
Branch on greater-than or eq.0

Branch on less than 0

Branch on greater than 0

C'è una ETICHETTA
che identifica
un'istruzione

Pseudocodice delle azioni sottostanti
PC += 4
if (\$s1 == \$s2) { PC += C }



Comparazione e salto condizionato

- `beq s1,s2,C` # Branch on equal
 - `bne s1,s2,C` # Branch on not equal
 - `bge s1,x0,C` # Branch on less-than or eq. 0
 - `bge x0,s1,C` # Branch on greater-than or eq.0
 - `blt s1,x0,C` # Branch on less than 0
 - `bgt s1,x0,C` # Branch on greater than 0
-
- `slt s0,s1,s2` # Set s0 to 1 if s1 l.than s2
 - `slti s0,s1,10` # Set s0 to 1 if s1 l.than 10

Esempio: max di 4 numeri

Codice C

Supponiamo che i valori siano in a, b, c, d
E che il risultato sarà in max

```
max = a;  
if (b>max) {  
    max = b;  
}  
if (c>max) {  
    max = c;  
}  
if (d>max) {  
    max = d;  
}
```

Assembly RISC

Supponendo che i valori siano nei registri
t0, t1, t2, t3
E che il risultato vada nel registro t4

```
    or t4, zero, t0  
CheckB:  
    blt t4, t1, UpdateB  
    j CheckC  
UpdateB:  
    or t4, zero, t1  
CheckC:  
    blt t4, t2, UpdateC  
    j CheckD  
UpdateC:  
    or t4, zero, t2  
CheckD:  
    blt t4, t3, UpdatedD  
    j End  
UpdatedD:  
    or t4, zero, t3  
End:
```



Assembly RISC-V

Direttive principali per l'assemblatore

- .data** definizione dei dati statici
- .text** definizione del programma

- .asciz** stringa terminata da \0
- .byte** sequenza di byte
- .double** sequenza di double
- .float** sequenza di float
- .half** sequenza di half words
- .word** sequenza di words

Codici mnemonici delle **istruzioni**

add, sub, beq ...

Codifica mnemonica dei **registri**

zero, sp, ra ... t0, s0

Etichette (per calcolare gli indirizzi relativi)

label:

L'assemblatore converte

il testo del programma in assembly
in codice macchina

e dalle etichette calcola gli indirizzi

Dei salti relativi

Delle strutture dati in memoria (offset)

Dei salti assoluti

NOTA: le **strutture di controllo del flusso** del programma vanno realizzate «a mano» usando i **salti condizionati** e le etichette